ISEN
ALL IS DIGITAL!
OUEST

yncréa

ALL IS DIGITAL

Projet - Big Data / IA / Web

Cholan Scheers
Noel Andy
Colsenet Ange CIPA4

Projet - Big Data / IA / Web

Étude du patrimoine arboré

Ange Colsenet
Andy Noel
Cholan Scheers

Sommaire

Introduction . . . 4
Partie IA: . . . 5
1. Besoin client 1 : Visualisation sur carte . . . 5
1.1. Objectif . . . 5
1.2. Préparation des données . . . 5
1.3. Apprentissage non-supervisé . . . 5
1.4. Évaluation du clustering . . . 6
1.5. Visualisation sur carte . . . 6
1.5.1. Carte des arbres - 2 Clusters . . . 7
1.5.2. Carte des arbres - 3 Clusters . . . 7
1.6. Script de prédiction . . . 8
1.7. Conclusion . . . 8
2. Besoin client 2 : Modèle de prédiction de l'âge . . . 8
2.1. Objectif . . . 8
2.1.1. Variable cible . . . 8
2.2. Préparation des données . . . 9
2.2.1. Analyse de la distribution de la variable cible . . . 9
2.2.2. Traitement des valeurs manquantes et aberrantes . . . 9
2.3. Développement et évaluation des modèles . . . 9
2.3.1. Stratégie d'entraînement . . . 10
2.3.2. Modèle de référence : Régression Logistique . . . 10
2.3.3. Decision Tree et Optimisation . . . 10
2.3.4. Modèle Final : Forêt Aléatoire (Random Forest) . . . 10
2.4. Analyse des résultats et performances . . . 10
2.4.1. Performance globale . . . 10
2.4.2. Matrice de confusion . . . 10
2.4.3. Analyse des scores Précision et Rappel . . . 11
2.4.4. Importance des variables . . . 12
2.5. Conclusion et Scripts . . . 12

2.5.1. Script d'exploitation : 12
2.5.2. Utilisation du script : 13
3. Besoin client 3 : Système d'alerte pour les tempêtes 13
3.1. Objectif 13
3.1.1. Variable cible 13
3.1.2. Features retenues 14
3.2. Prétraitement des données 14
3.2.1. Transformation des variables numériques 14
3.2.2. Gestion des valeurs manquantes 14
3.2.3. Encodage des variables catégorielles 15
3.2.4. Split train / test 15
3.3. Modélisation 15
3.3.1. Choix de l'algorithme 15
3.3.2. Seuil de décision 15
3.4. Résultats et évaluation 16
3.4.1. Métriques 16
3.4.2. Lecture des résultats 16
3.5. Scripts de prédiction 17
Annexes 18

Introduction

Ce rapport présente le projet de Big Data/IA/Web visant à

Le projet porte sur le développement d'une application d'étude du patrimoine arboré de la ville de Saint-Quentin (Aisne).

Le travail est structuré en trois parties principales : Big Data, Intelligence Artificielle et Développement Web, chacune représentant un volume de travail équivalent. Bien que ces parties soient traitées de manière indépendante, elles contribuent toutes à une application globale cohérente.

Partie IA:

1. Besoin client 1 : Visualisation sur carte

1.1. Objectif

L'objectif de ce besoin est d'afficher sur une carte les différents arbres plantés dans la ville de Saint-Quentin selon leurs tailles. Pour cela, nous utilisons une IA qui s'occupe d'étudier les données puis de trier les arbres en fonction de qu'ils sont grands ou petits, avec deux clusters, ou alors s'ils sont grands, moyens ou petits, avec trois clusters.

Cela permet de mieux visualiser la répartition des arbres dans la ville.

1.2. Préparation des données

Pour permettre à notre IA d'étudier correctement les différents arbres, nous devons lui fournir une base de données nettoyée et préparée. Cela a été effectué durant le précédent TP BIG DATA, où nous avons retiré les doublons, et les données aberrantes.

Ensuite, nous utilisons trois colonnes de notre fichier CSV :

- X pour la longitude
- Y pour la latitude
- haut_tot pour avoir la hauteur totale de l'arbre

Également, pour rajouter un tri pour mon objectif, nous avons supprimé les arbres ayant des valeurs manquantes ou des hauteurs nulles. Cela permet de garder les arbres plantés ou les données cohérentes pour le traitement de ces dernières.

De plus, pour l'apprentissage non-supervisé, nous avons gardé qu'une seule donnée pour réaliser l'étude, la hauteur totale. Nous n'avons pas gardé par exemple le diamètre, qui ne permettait pas d'avoir une cohérence dans le tri des arbres.

1.3. Apprentissage non-supervisé

Pour regrouper les arbres selon leur taille, nous avons utilisé un algorithme de clustering.

Nous avons choisi la méthode des K-means car cela est idéal pour notre utilisation : adapté aux données numériques, simple d'utilisation et permet de fixer directement le nombre de groupes.

Deux cas ont été étudiés :

- k = 2 (petit / grand)
- k = 3 (petit / moyen / grand)

Les clusters obtenus ont ensuite été triés selon la hauteur moyenne afin d'attribuer un sens aux groupes.

1.4. Évaluation du clustering

Afin d'évaluer la qualité des clusters mis en place, trois métriques ont été utilisées :

- Silhouette Coefficient : mesure la cohérence des groupes
- Calinski-Harabasz Index : mesure la séparation entre les clusters
- Davies-Bouldin Index : mesure le chevauchement des clusters

Les résultats obtenus permettent de montrer la bonne séparation et la cohérence entre les clusters. Cela permet de confirmer que la hauteur est une variable pertinente pour distinguer les tailles des arbres.

1.5. Visualisation sur carte

Les arbres ont été représentés sur une carte grâce à la bibliothèque Plotly.

Les coordonnées ont été converties en format GPS afin d'être affichées correctement sur la carte générée.

Selon le choix du nombre de clusters, deux cartes peuvent être générées. L'une avec trois couleurs pour le choix de petit / moyen / grand et l'autre carte avec deux couleurs pour petit / grand.

Chaque arbre est affiché avec :

- Une couleur correspondant à sa catégorie (petit, moyen, grand ou petit, grand)
- Un affichage interactif permettant de consulter ses informations

Cette visualisation permet d'observer la répartition spatiale des arbres selon leur taille dans la ville de Saint-Quentin de manière simple et compréhensible par tous.

1.5.1. Carte des arbres - 2 Clusters

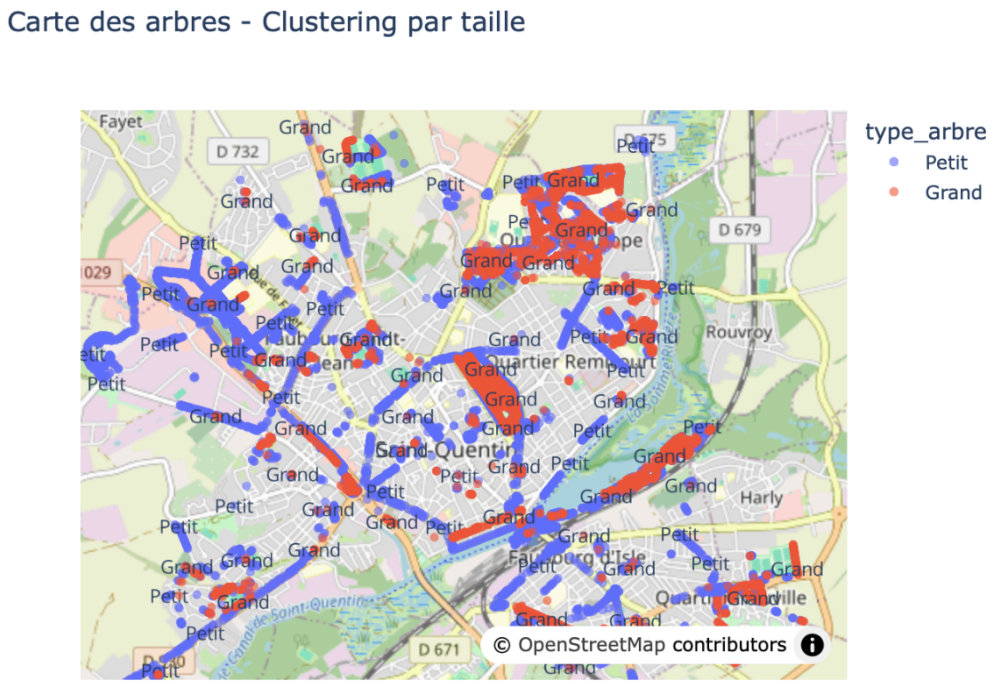

Figure 1: Carte

,

1.5.2. Carte des arbres - 3 Clusters

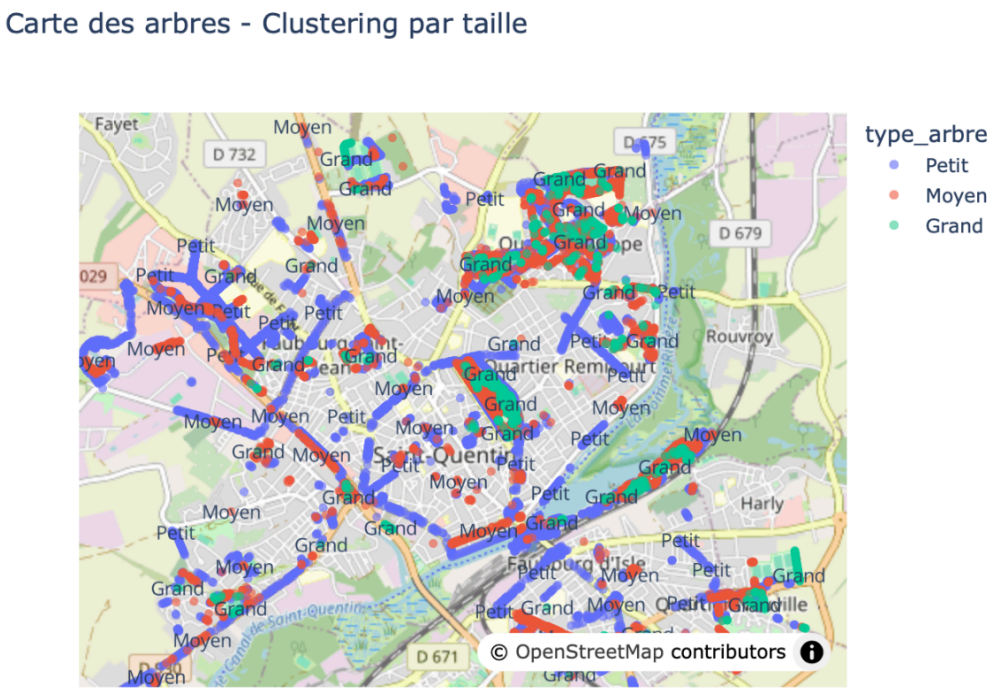

Figure 2: Matrice de confusion

,

1.6. Script de prédiction

En plus du premier script permettant l'affichage de la répartition des arbres, un modèle est généré permettant ensuite de prédire un arbre en fonction de sa hauteur.

Ce script charge le modèle préalablement entraîné, ne relance pas l'apprentissage, ce qui permet un gain de temps et une cohérence avec les données initiales.

Il retourne ensuite le cluster et le type d'arbre associé. Cela permet donc une utilisation simple et rapide du modèle généré dans le premier script.

1.7. Conclusion

Le clustering permet de regrouper de manière efficace les arbres selon leur taille sans avoir besoin de réalisé un étiquetage de données.

La visualisation sur carte apporte une dimension supplémentaire en permettant une analyse de la répartition géographique des arbres à Saint-Quentin.

2. Besoin client 2 : Modèle de prédiction de l'âge

2.1. Objectif

Le Client 2 souhaite estimer l'âge des arbres de Saint-Quentin.

2.1.1. Variable cible

La base de données initiale fournit une donnée nommée `age_estim`, mais celle-ci est parfois incomplète ou difficile à vérifier.

L'enjeu est de pouvoir déduire l'âge d'un spécimen à partir de ses caractéristiques physiques mesurables telles que :

- La hauteur totale et la hauteur du tronc.
- Le diamètre du tronc.
- Le stade de développement apparent.
- Le nom latin de l'espèce

Plutôt que de prédire une valeur numérique exacte (régression), le modèle utilisera la classification multi-classe. L'objectif est de classer chaque arbre dans l'une des quatre catégories suivantes (Jeune, Jeune Adulte, Adulte, Vieux)

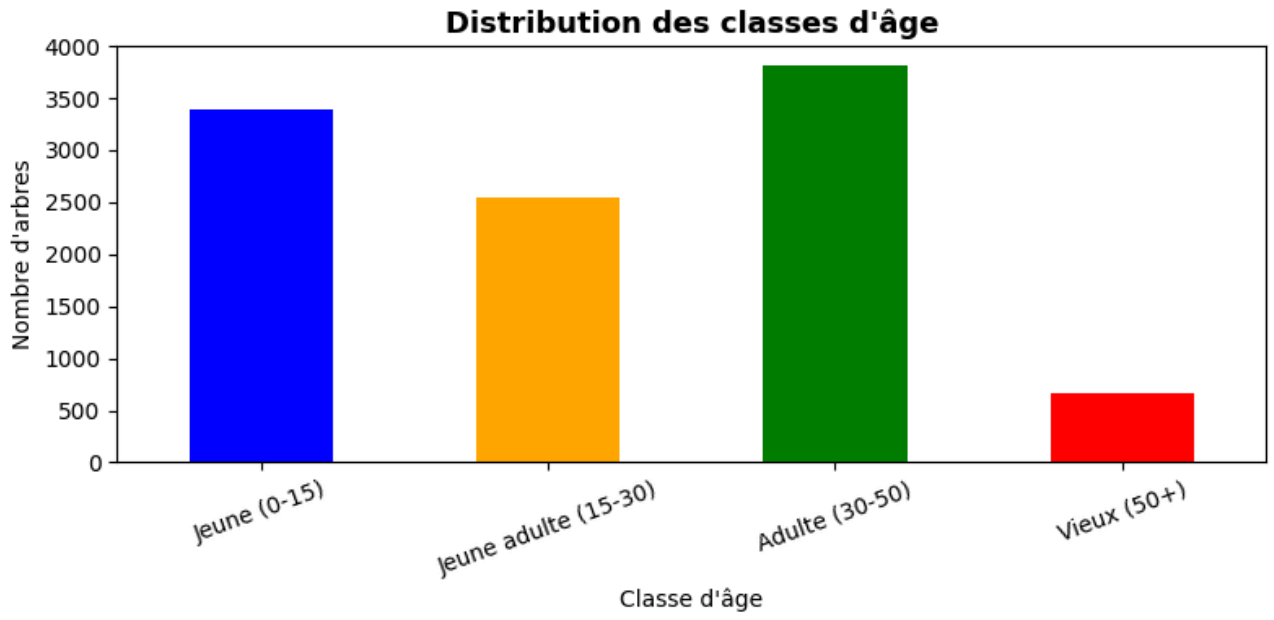

Figure 3: Distribution des classes

,

2.2. Préparation des données

2.2.1. Analyse de la distribution de la variable cible

La variable `age_estim` possède une distribution hétérogène. Afin de faciliter l'apprentissage, nous avons procédé à une transformation de cette variable continue en classes catégorielles.

2.2.2. Traitement des valeurs manquantes et aberrantes

Conformément aux observations du notebook, les étapes suivantes ont été appliquées :

- **Imputation** : Utilisation de la médiane pour combler les valeurs manquantes sur les variables numériques (`haut_tot`, `tronc_diam`) et utilisation de la valeur la plus fréquente pour les variables qualitatives
- **Filtrage** : Supression des arbres n'ayant pas d'âge
- **Ajout de variable** : Pour mieux interpréter les valeurs de diamètre et de hauteur du tronc une nouvelle valeur ratio diamètre/hauteur a été ajoutée au dataset.

2.3. Développement et évaluation des modèles

L'objectif de cette phase est de comparer différentes techniques d'apprentissage supervisé pour identifier celle qui généralise le mieux la prédiction des classes d'âge.

2.3.1. Stratégie d'entraînement

Pour garantir la robustesse des résultats, nous avons utilisé un découpage classique des données en ensemble d'entraînement et de test ($80\frac{\%}{20}\%$). Une **Stratified K-Fold** a été privilégiée lors de la validation croisée afin de maintenir la proportion des classes d'âge dans chaque échantillon.

2.3.2. Modèle de référence : Régression Logistique

Nous avons débuté par une **Régression Logistique** servant de cas générique. Malgré sa simplicité, ce modèle linéaire permet de vérifier si les variables (diamètre, hauteur) présentent une corrélation directe et simple avec l'âge.

2.3.3. Decision Tree et Optimisation

L'utilisation d'un **DecisionTreeClassifier** a permis de capturer des relations non-linéaires. Pour limiter le sur-apprentissage, nous avons procédé à une recherche d'hyperparamètres via **RandomizedSearchCV**, en agissant notamment sur :

- `max_depth` : Profondeur maximale de l'arbre.
- `min_samples_split` : Nombre minimum d'échantillons pour diviser un nœud.

2.3.4. Modèle Final : Forêt Aléatoire (Random Forest)

Le modèle retenu est **Random Forest**. En combinant plusieurs arbres de décision, cette méthode réduit significativement la variance des erreurs.

2.4. Analyse des résultats et performances

L'évaluation de notre modèle de classification final s'appuie sur une analyse multi-critère afin de valider sa capacité à distinguer les quatre phases de vie des arbres.

2.4.1. Performance globale

Le modèle Random Forest affiche une précision globale satisfaisante. Toutefois, l'analyse détaillée par classe révèle des disparités de performance sur les classes extrêmes ("Vieux" notamment) qui disposent de moins d'échantillons.

2.4.2. Matrice de confusion

La matrice de confusion nous permet d'observer les erreurs de classement. On remarque que les erreurs se font majoritairement vers les classes adjacentes (par exemple, un "Jeune Adulte" classé en "Adulte").

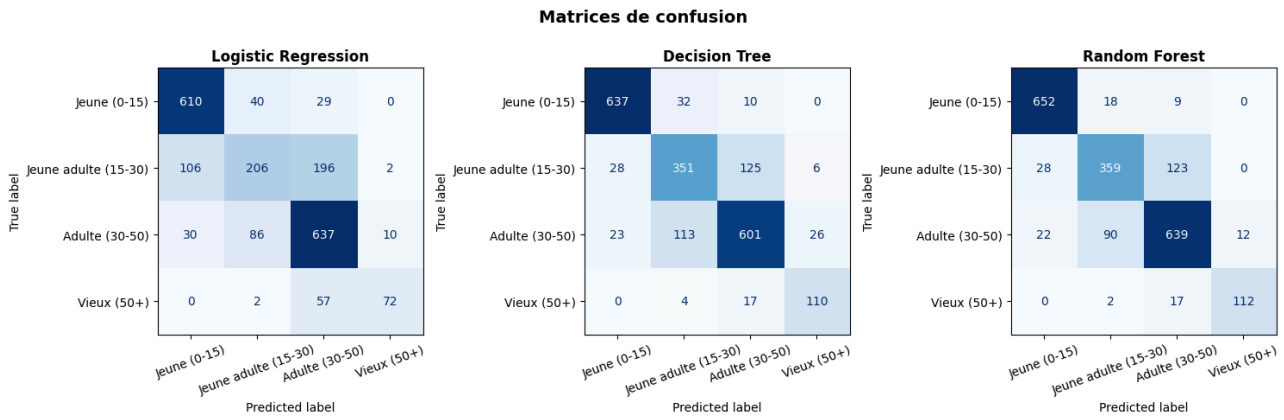

Figure 4: Matrice de confusion

,

2.4.3. Analyse des scores Précision et Rappel

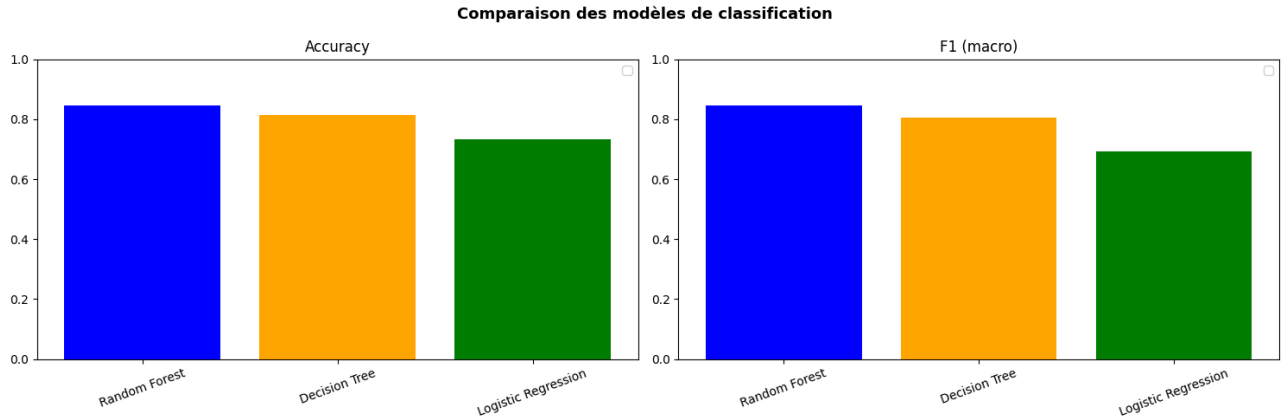

Figure 5: Performance relative du modèle final

,

- Random Forest (Bleu) : Le plus performant avec un score d'environ 84 %.
- Decision Tree (Orange) : Performance intermédiaire, proche de 80 %.
- Logistic Regression (Vert) : Le moins performant, se situant autour de 69 %.

Le Random Forest et le Decision Tree surpassent nettement la Régression Logistique. Cela confirme que les relations entre vos variables (diamètre, hauteur, stade développement etc.) et l'âge de l'arbre ne sont pas purement linéaires, ce qui explique pourquoi la régression logistique peine à capturer toute la complexité du phénomène.

2.4.4. Importance des variables

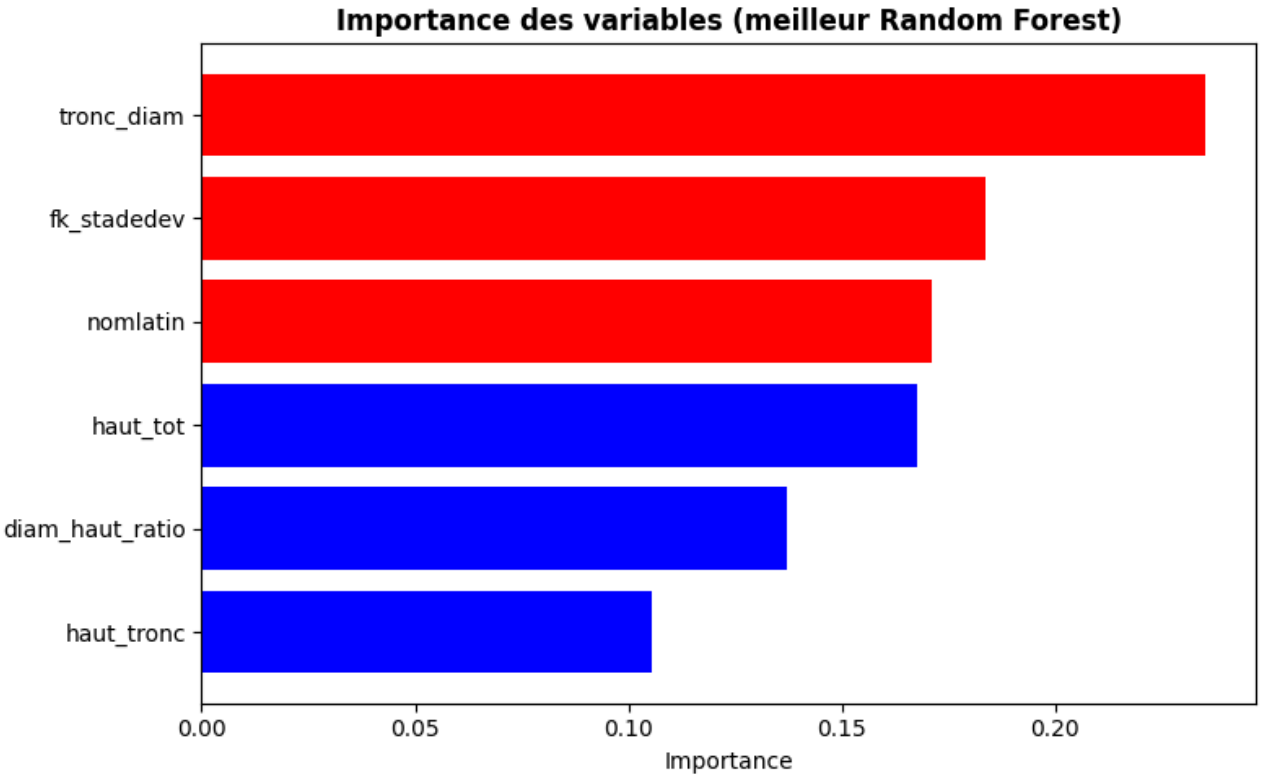

Figure 6: Importance des caractéristiques

,

L'analyse de l'importance des caractéristiques montre que le diamètre du tronc est le facteur le plus déterminant dans la prédiction de l'âge, suivi par le stade de développement, le nom latin, la hauteur totale, le ratio diamètre/hauteur puis le hauteur du tronc.

2.5. Conclusion et Scripts

L'étude menée pour le Client 2 démontre qu'il est possible d'estimer avec une fiabilité satisfaisante la catégorie d'âge d'un arbre à partir de données dendrométriques simples (hauteur et diamètre). Le modèle final de Random Forest offre un équilibre robuste entre précision et capacité de généralisation.

2.5.1. Script d'exploitation :

Fonctionnalités du script :

- **Chargement du modèle** : Il importe le modèle complet (`StandardScaler` + `RandomForest`) sauvegardé au format `.pkl` lors de la phase d'entraînement.
- **Traitement des entrées** : Le script accepte les mesures brutes de l'arbre, calcule automatiquement le ratio diamètre/hauteur (caractéristique clé identifiée lors de l'analyse) et applique les transformations nécessaires.

- **Résultat instantané** : Il retourne la classe d'âge prédite ainsi que l'indice de confiance associé à la prédiction.

2.5.2. Utilisation du script :

Pour utiliser le script il suffit de lancer le terminal au niveau du fichier script et de lancer la commande :

```
python script.py --haut_tot 12.0  --haut_tronc 2.5 --tronc_diam 150 --
fk_stadedev adulte --nomlatin TILCOR
```

3. Besoin client 3 : Système d'alerte pour les tempêtes

3.1. Objectif

L'objectif de cette partie était de développer un modèle d'intelligence artificielle basé sur un apprentissage supervisé par classification, afin de prédire le risque de déracinement des arbres lors d'événements tempétueux.

Pour cela, nous avons utilisé le jeu de données préalablement nettoyé, afin d'entraîner notre modèle sur des données pertinentes pour la phase d'apprentissage. L'enjeu principal de cet apprentissage était d'identifier les variables les plus exploitables permettant d'obtenir une classification fiable.

Cependant, le nombre de données liées spécifiquement au déracinement dû aux tempêtes était très limité, voire inexistant. Nous avons donc adapté notre approche en cherchant à prédire le risque qu'un arbre soit abattu ou essouché, c'est-à-dire la probabilité qu'il ne soit plus en place en fonction de ses caractéristiques.

3.1.1. Variable cible

La colonne `fk_arb_etat` décrit l'état de chaque arbre et constitue la variable cible que nous cherchons à prédire.

Étant donné que l'objectif est de déterminer si un arbre présente un risque de ne plus être en place, nous avons formulé le problème sous forme d'une classification binaire, définie comme suit :

Classe	États inclus	Effectif
0 — EN PLACE	EN PLACE	~ 9 200
1 — RISQUE	ABATTU, Essouché, SUPPRIMÉ, REMPLACÉ, Non essouché	~ 1 000

de par les effectif des données, on vois que le déséquilibre est d'environ **10:1**.
Pour la suite de l'apprentissage on mettra donc un poid de 10:1 pour la classe **Risque**

3.1.2. Features retenues

Les features ont été choisies pour leur pertinence physique vis-à-vis de la résistance au vent et de la fragilité structurelle de l'arbre.

Feature	Type	Justification
`tronc_diam`	Num.	Résistance mécanique au vent
`haut_tot`	Num.	Prise au vent
`haut_tronc`	Num.	Position du centre de gravité
`age_estim`	Num.	Fragilité potentielle liée à l'âge
`X`, `Y`	Num.	Localisation géographique
`fk_port`	Cat.	Forme générale — prise au vent
`fk_pied`	Cat.	Fragilité racinaire
`nomfrancais`	Cat.	Espèce — résistance intrinsèque
`clc_quartier`	Cat.	Contexte urbain d'exposition
`clc_secteur`	Cat.	Secteur géographique

les aspect géographique on également été ajouté après avoir constaté qu'il permettait d'amélioré significativement l'apprentissage, on a donc également:

Feature	Type	Justification
`X`, `Y`	Num.	Localisation géographique
`clc_quartier`	Cat.	Contexte urbain d'exposition
`clc_secteur`	Cat.	Secteur géographique

3.2. Prétraitement des données

Le prétraitement suit les recommandations de la documentation scikit-learn (`sklearn.preprocessing`) et est entièrement encapsulé dans un **Pipeline** pour éviter toute fuite de données entre l'entraînement et le test.

3.2.1. Transformation des variables numériques

Une **transformation logarithmique** (`log1p`) est appliquée avant le pipeline sur les variables numériques afin de réduire l'asymétrie des distributions (diamètre, hauteur, âge présentent typiquement une distribution à longue queue droite).

Ensuite, un **StandardScaler** normalise chaque variable (moyenne = 0, écart-type = 1), ce qui est nécessaire pour que des variables d'unités différentes (cm, m, années) contribuent de façon égale au modèle.

3.2.2. Gestion des valeurs manquantes

Un **SimpleImputer** est appliqué :

- on prend la **médiane** pour les variables numériques manquante
- pour les catégorie manquante on set a (`"missing"`) pour les variables catégorielles

3.2.3. Encodage des variables catégorielles

Un **OneHotEncoder** transforme chaque modalité en colonne binaire. Le paramètre `handle_unknown="ignore"` permet de gérer proprement les modalités inconnues lors de la prédiction sur de nouvelles données.

3.2.4. Split train / test

Le jeu est divisé en **80% entraînement / 20% test** avec `stratify=y` pour garantir les mêmes proportions de classes dans chaque partition du fait qu'on est des données déséquilibré.

3.3. Modélisation

3.3.1. Choix de l'algorithme

Un Random Forest Classifier (ensemble de 500 arbres de décision) a été retenu après avoir testé plusieurs modèles, notamment XGBoost, CatBoost et Random Forest. Ce choix s'explique par sa capacité à mieux minimiser le nombre de faux négatifs tout en maximisant la détection des arbres à risque.

Cependant, les trois algorithmes présentent des performances globales similaires en termes d'accuracy :

- Accuracy XGBoost : 92 %
- Accuracy CatBoost : 93,91 %
- Accuracy Random Forest : 92,27 %

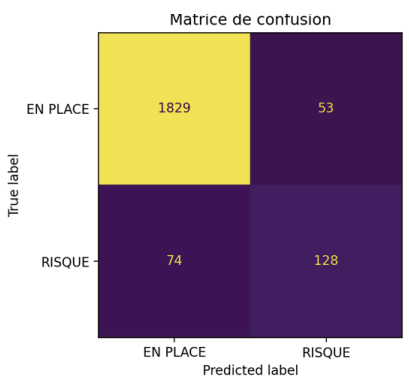

Figure 7: Via CatBoost

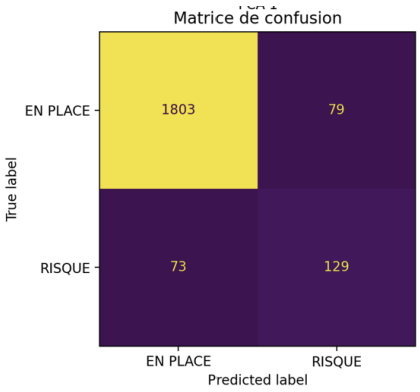

Figure 8: Via XGB

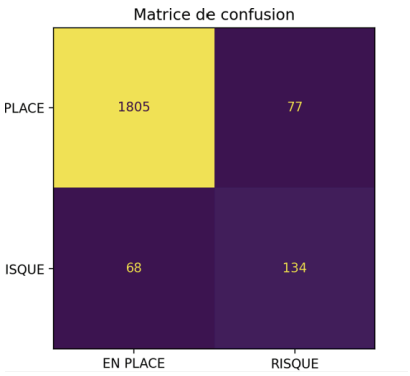

Figure 9: Via CatBoost

3.3.2. Seuil de décision

Par défaut, sklearn prédit la classe 1 dès que la probabilité dépasse 0.5. Après exploration des seuils, **0.2** a été retenu : pour maximiser le rappel qu'on fais sur la classe

RISQUE afin d'être cohérent avec le fais qu'on cherche a prédire qu'elle arbre est a risque lors d'une tempête. dans ce cas la, on préfère prédire plus de Faux risque que de manqué des arbres a risque qui pourrais être dangereux.

3.4. Résultats et évaluation

3.4.1. Métriques

Classe	Precision	Recall	F1
EN PLACE (0)	0.97	0.96	0.96
RISQUE (1)	0.63	0.68	0.65
Accuracy			0.93
Macro avg	0.80	0.82	0.81

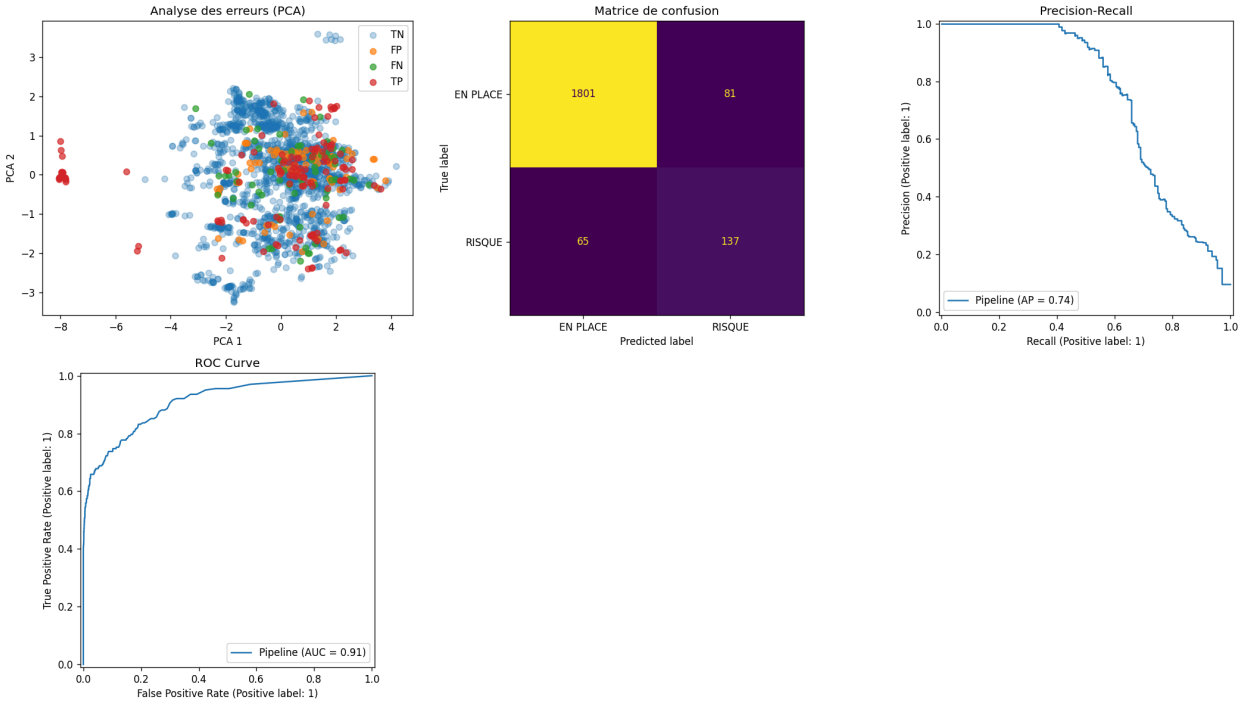

Figure 10: Performance relative du modèle final

,

3.4.2. Lecture des résultats

La **précision de 63%** pour la prédiction de la classe RISQUE signifie que sur 10 alertes déclenchées, environ 6 correspondent à de vrais arbres à risque.

La **matrice de confusion** confirme que 137 arbres à risque sont correctement détectés sur 202, on a 81 fausses alarmes (81) pour 137 arbre a risque bien prédit ce qui reste acceptable

La **courbe Precision-Recall** (AP = 0.74) montre que le modèle maintient une bonne précision jusqu'à un rappel de 0.6, ce qui est satisfaisant au regard du déséquilibre initial des données.

3.5. Scripts de prédiction

le scripts **`predict_ui.py`** exploite le modèle sauvegardé (`modele_tempete.pkl`) sans relancer d'entraînement afin de prédire si un arbre est a risque en fonction de ces caractéristique.

Une interface graphique a été réalisé en utilisant QT afin de "simuler" un usage pratique

En utilisant une boucle pour testé toutes les données du CSV utilisé pour l'entrainement on trouve que la moyenne des arbre a abbatre est ~ 9% ce qui reste cohérent avec un nombre d'arbre potentiellement sensible à une tempête.

Annexes

Figure 1 Carte 7
Figure 2 Matrice de confusion 7
Figure 3 Distribution des classes 9
Figure 4 Matrice de confusion 11
Figure 5 Performance relative du modèle final 11
Figure 6 Importance des caractéristiques 12
Figure 7 Via CatBoost 15
Figure 8 Via XGB 15
Figure 9 Via CatBoost 15
Figure 10 Performance relative du modèle final 16

L'ISEN est une grande école d'ingénieurs présente sur 4 sites (1800 étudiants sur les campus de Brest, Lille, Nantes, Rennes et Toulon). Elle est habilitée par la Commission des Titres d'Ingénieur (CTI) et est membre de la Conférence des Grandes Ecoles (CGE).

Depuis près de 60 ans, l'ISEN forme des ingénieurs généralistes de haut niveau dans les domaines du numérique et de ses applications.

ALL IS DIGITAL !

isen-brest.fr
isen-nantes.fr
isen-rennes.fr

ISEN
ALL IS DIGITAL!

yncréa

Réalisation : www.APPALOOSA.fr - 02 98 79 81 00 - Crédits photos : istockphoto, Joachim Dassonville - 2015/2016 - Document non contractuel